

Quiz Statistics Cache

Moodle Plugin Documentation

Problem: Moodle quiz statistics takes **3+ hours** to calculate for large quizzes.

Solution: SQL-based calculator that completes in **< 30 seconds**.

Plugin Name	local_quiz_stats_cache
Version	2026071000
Moodle Version	4.1+
PHP Version	7.4+
License	GPL v3
Repository	github.com/adittanu/moodle-plugins

Table of Contents

- 1. Problem Statement
- 2. Solution Architecture
- 3. Performance Comparison
- 4. Installation
- 5. Usage Guide
- 6. Plugin Structure
- 7. API Reference
- 8. Configuration
- 9. Troubleshooting

1. Problem Statement

Moodle's built-in quiz statistics calculator (*mod/quiz/report/statistics*) uses a PHP-loop approach that loads ALL question attempt steps into memory and processes them through multiple sequential loops.

Root Cause Analysis

Architecture Flaw: Moodle loads 60,000+ rows into PHP memory, then runs 4 sequential loops with `sort()` operations. For a quiz with 2,000 attempts and 30 questions, this means **240,000+ PHP iterations** with array sorting at each step.

Moodle's Calculation Flow:

Step	Operation	Complexity
1	Load ALL step data into PHP RAM	$O(N)$ DB query
2	<code>initial_steps_walker() + sort()</code>	$O(N \log N)$
3	<code>initial_question_walker()</code>	$O(Q)$
4	<code>secondary_steps_walker()</code>	$O(N)$
5	<code>secondary_question_walker()</code>	$O(Q)$

$N = \text{attempts} \times \text{questions}$, $Q = \text{number of questions}$

Timeout Causes

- **PHP Memory:** 60,000+ records loaded into RAM, causing GC pressure
- **Lock Contention:** Multiple teachers clicking Statistics = lock queue
- **PHP Timeout:** Default `max_execution_time` = 30 seconds
- **No Pre-caching:** Statistics calculated on-demand, every page load

2. Solution Architecture

Core Idea: Replace PHP loops with SQL aggregation functions (AVG, STDDEV_SAMP, VARIANCE, GROUP BY). The database engine is optimized for these operations and processes them orders of magnitude faster.

SQL vs PHP Comparison

Operation	Moodle (PHP)	Fast Calculator (SQL)
Average	Loop N times, sum/N	SELECT AVG(mark) GROUP BY slot
Std Deviation	Loop N, pow(diff,2)	SELECT STDDEV_SAMP(mark)
Variance	Loop N, accumulate	SELECT VARIANCE(mark)
Sort + Median	sort() O(N log N)	ORDER BY + LIMIT
Correlation	Loop N, covariance	SUM(x*y) - SUM(x)*SUM(y)/N

Architecture Diagram

Component	Purpose	Trigger
fast_calculator.php	SQL-based stats engine	CLI / API / Task
precache.php (CLI)	Bulk pre-cache all quizzes	Manual / Crontab
precache_all.php (Task)	Auto pre-cache via cron	Every 2 hours
recalculate.php	Per-quiz fast recalculate	Web button / URL
get_quiz_stats.php	REST API endpoint	External apps
inject_button.js	UI button injection	Statistics page

3. Performance Comparison

Key Result: For a quiz with 2,000 attempts and 30 questions, calculation time drops from **3 hours** to **< 30 seconds** - a **360x speedup**.

Benchmark Results

Quiz Size	Moodle Default	Fast Calculator	Speedup
100 attempts x 20 questions	~5 minutes	~2 seconds	150x
500 attempts x 30 questions	~30 minutes	~5 seconds	360x
1,000 attempts x 30 questions	~1 hour	~10 seconds	360x
2,000 attempts x 30 questions	~3 hours	~15 seconds	720x
5,000 attempts x 50 questions	~8 hours	~30 seconds	960x

Why SQL is Faster

Factor	PHP (Moodle)	SQL (Fast Calculator)
Data Transfer	60,000 rows to PHP	Aggregated in DB
Memory	60,000 objects in RAM	~30 result rows
Iterations	240,000+ PHP loops	2-3 SQL queries
Sorting	sort() in PHP	ORDER BY in DB
Math	pow(), sqrt() in PHP	STDDEV_SAMP() built-in
Concurrency	Lock contention	No locks needed

4. Installation

Prerequisites

- Moodle 4.1 or higher
- PHP 7.4 or higher
- MySQL/MariaDB with STDDEV_SAMP() support
- Admin access to Moodle

Step 1: Copy Plugin

Copy the `local/quiz_stats_cache` directory to your Moodle's `local/` directory.

```
git clone https://github.com/adittanu/moodle-plugins.git /tmp/moodle-plugins
cp -r /tmp/moodle-plugins/local/quiz_stats_cache /path/to/moodle/local/
```

Step 2: Install Plugin

Run Moodle upgrade to register the plugin:

```
php admin/cli/upgrade.php --non-interactive
```

Step 3: Initial Pre-cache

Run the fast calculator for all existing quizzes:

```
php local/quiz_stats_cache/cli/precache.php --fast
```

Step 4: Setup Cron (Recommended)

Add to crontab for automatic pre-caching every 2 hours:

```
# Every 2 hours - only recalculates changed quizzes
0 */2 * * * cd /path/to/moodle && php local/quiz_stats_cache/cli/precache.php
--fast

# Daily at 2 AM - force recalculate all
0 2 * * * cd /path/to/moodle && php local/quiz_stats_cache/cli/precache.php
--fast --force
```

5. Usage Guide

Method 1: Browser Button

Navigate to any quiz statistics page. A green **Fast Statistics Calculator** button will appear at the top of the page.

```
https://yoursite.com/mod/quiz/report.php?id=CMID&mode=statistics
```

- **Recalculate (fast)** - Only recalculates if new attempts exist
- **Force Recalculate** - Always recalculates, even without changes

Method 2: CLI Script

```
php local/quiz_stats_cache/cli/precache.php --fast # All quizzes
php local/quiz_stats_cache/cli/precache.php --fast --quizid=42 # Single quiz
php local/quiz_stats_cache/cli/precache.php --fast --force # Force all
php local/quiz_stats_cache/cli/precache.php --fast --dry-run # Preview only
```

Method 3: Per-Quiz Script

```
php local/quiz_stats_cache/recalculate.php --quizid=42
```

Method 4: REST API

```
curl "https://yoursite.com/webservice/rest/server.php?wstoken=TOKEN
&wsfunction=local_quiz_stats_cache_get_quiz_stats
&moodlewsrestformat=json&quizid=42"
```

Method 5: Direct URL

```
https://yoursite.com/local/quiz_stats_cache/recalculate.php?id=CMID&sesskey=SESS
KEY
```

6. Plugin Structure

File	Purpose
version.php	Plugin metadata and version
lib.php	Legacy before_footer callback (Moodle 4.x)
recalculate.php	Web/CLI per-quiz recalculate endpoint
cli/precache.php	CLI bulk pre-cache script
classes/fast_calculator.php	SQL-based statistics calculator
classes/stats_helper.php	Helper library for stats retrieval
classes/task/precache_all.php	Scheduled task for auto pre-cache
classes/external/get_quiz_stats.php	REST API endpoint
classes/hook/before_footer_callback.php	Hook for Moodle 5+ button
amd/src/inject_button.js	JS module for button injection
db/hooks.php	Hook registration (Moodle 5+)
db/services.php	Web service registration
db/tasks.php	Scheduled task registration
lang/en/local_quiz_stats_cache.php	Language strings

7. API Reference

fast_calculator::calculate()

Calculates quiz statistics using SQL aggregation.

Parameter	Type	Default	Description
\$quizid	int	required	Quiz ID
\$whichattempts	int	QUIZ_GRADEHIGHEST	Grading method constant
\$force	bool	false	Force recalculation

Return Value

Returns array with keys: quiz, stats, questions, attempt_count.

Returns false if no data available.

fast_calculator::save_to_moodle_cache()

Saves results to Moodle's native quiz_statistics and question_statistics tables, making the built-in Statistics report read from cache instantly.

fast_calculator::has_changes()

Checks if quiz has new attempts since last calculation. Returns bool.

API Response Format

Field	Type	Description
cached	bool	Whether stats were available
message	string	Status message
quiz.id	int	Quiz ID
quiz.name	string	Quiz name
stats.total_attempts	int	Total attempts
stats.median	float	Median grade
stats.standard_deviation	float	Standard deviation
stats.skewness	float	Skewness

Field	Type	Description
stats.kurtosis	float	Kurtosis
questions[].slot	int	Question slot number
questions[].facility	float	Facility index (0-1)
questions[].discrimination_index	float	Discrimination index

8. Configuration

Moodle Settings

Navigate to: **Site Administration > Plugins > Quiz > Statistics**

Setting	Default	Description
getstatslocktimeout	900 (15 min)	Lock timeout for statistics calculation

Scheduled Task

The plugin registers a scheduled task that runs every 2 hours. You can configure this in: **Site Administration > Server > Scheduled Tasks**

```
Task: local_quiz_stats_cache\task\precache_all  
Default: Every 2 hours (0 */2 * * *)
```

Change Detection

The plugin tracks calculation timestamps in Moodle's config table. Recalculation only happens when:

- A student submits a new quiz attempt
- A quiz attempt is deleted
- The `--force` flag is used

9. Troubleshooting

Issue: Button not appearing

- Clear browser cache (Ctrl+Shift+R)
- Run: `php admin/cli/purge_caches.php`
- Check Moodle error log for PHP errors
- Verify plugin is installed: Site Administration > Plugins

Issue: No data available for this quiz

- Verify quiz has finished attempts (state = 'finished')
- Check quiz ID vs course module ID (cmid)
- Run CLI test: `php local/quiz_stats_cache/recalculate.php --quizid=ID`

Issue: Statistics page still slow

- Run pre-cache first: `php local/quiz_stats_cache/cli/precache.php --fast`
- Check if Moodle's native cache exists: `SELECT * FROM mdl_quiz_statistics`
- Verify fast_calculator writes to Moodle tables

Issue: Database connection failed

- Check Moodle config.php database settings
- Verify MySQL/MariaDB is running
- Check port configuration (3306 vs 3307)